

Absolutely. For **Software Engineering (SE), CS24353**, I'll follow the **same system as the previous subjects**: complete M1–M5 coverage, no syllabus topic skipped, exam-oriented explanations, memory hooks, active recall, comparisons, diagrams, and PYQ-oriented questions.

I checked your actual BIT Mesra course document. SE is a **3-credit course**, and the official syllabus is divided into five modules.

SOFTWARE ENGINEERING — COMPLETE MASTER NOTES

CS24353 | M1–M5 | 67 Topics

The study guide you uploaded expands the official syllabus into **67 individual study topics**.

MODULE I — INTRODUCTION TO SOFTWARE ENGINEERING

13 Topics

Introduction to Software Engineering

Definitions & FAQs

Evolving Role of Software

Software Process Models

Waterfall Model

Prototyping Model

Spiral Model

RAD Model

Incremental Model

Agile Models

Management Activities

Project Planning

Project Scheduling

Risk Management

Note: The study guide groups process models under one heading but expands them into individual models; therefore the checklist counts this module as **13 study areas**.

1. What is Software Engineering?

Software Engineering (SE) is the systematic, disciplined and measurable approach to the development, operation and maintenance of software.

Think:

```
Programming
↓
Writing code

Software Engineering
↓
Requirements
↓
Planning
↓
Design
↓
Implementation
↓
Testing
↓
Deployment
↓
Maintenance
```

Memory

Key Insight

Software Engineering = Engineering principles applied to software.

The official course objective specifically emphasizes software process models, project management, requirements, design, quality, estimation, maintenance and evolution.

2. Why Software Engineering?

Large software systems create problems involving:

complexity
changing requirements
cost
deadlines
reliability
maintainability
security
team coordination

Without engineering discipline:

```
Poor requirements
↓
Poor design
↓
More bugs
↓
More rework
↓
Higher cost
↓
Project failure
```

3. Evolving Role of Software

Software has evolved from simple programs into systems that control:

communication

banking

transportation

healthcare

manufacturing

entertainment

cloud systems

embedded systems

AI systems

Important idea

Modern software is:

Key Insight

a product + a service + an evolving system

Software doesn't simply "finish."

It continuously changes.

4. Software Process Model

A **software process model** defines how software development activities are organized.

Common activities:

```
Requirements
  ↓
Design
  ↓
Implementation
  ↓
Testing
  ↓
Deployment
  ↓
Maintenance
```

Different models arrange these activities differently.

5. Waterfall Model

Sequential development model.

```
Requirements
↓
Design
↓
Implementation
↓
Testing
↓
Deployment
↓
Maintenance
```

Advantages

simple
structured
easy to manage
documentation-heavy

Disadvantages

difficult to accommodate changes
testing occurs relatively late
poor fit for uncertain requirements

Best suited

Stable, well-understood requirements.

6. Prototyping Model

Build an early prototype to understand requirements.

```
Initial Requirements
↓
Prototype
↓
User Feedback
↓
Refined Requirements
↓
Final System
```

Useful when:

Key Insight

customer doesn't know exactly what they want.

7. Spiral Model

Combines iterative development with **risk analysis**.

Each cycle generally involves:

```
Objectives
↓
Risk Analysis
↓
Development
↓
Evaluation
↓
Next Spiral
```

Memory

Key Insight

Spiral = Iteration + Risk

Best for large, complex and high-risk projects.

8. RAD

Rapid Application Development

Focus:

Key Insight

fast development + reusable components + user feedback.

Typical characteristics:

rapid prototyping

component reuse

iterative development

strong user involvement

9. Incremental Model

Build the system in multiple increments.

```
Increment 1 → Core features
Increment 2 → More features
Increment 3 → More features
Increment 4 → Complete product
```

Difference from waterfall

Waterfall:

Key Insight

one major sequential development path.

Incremental:

Key Insight

working versions delivered progressively.

10. Agile Models

Agile emphasizes:

iterative development
customer collaboration
responding to change
frequent delivery
working software
continuous feedback

Agile cycle

```
Plan
↓
Develop
↓
Test
↓
Review
↓
Feedback
↻
```

🧠 Agile memory

Key Insight

Build → Show → Learn → Adapt

Process Model Comparison

Model	Main idea	Best when
Waterfall	Sequential	Stable requirements
Prototype	Discover requirements	Requirements unclear
Spiral	Risk-driven iteration	High-risk projects
RAD	Rapid development	Fast delivery
Incremental	Feature increments	Early partial delivery
Agile	Adaptive iterations	Changing requirements

🔥 Exam question

Compare Waterfall, Spiral, Prototyping, Incremental and Agile models.

This is a perfect long-answer question.

11. Management Activities

Major software management activities:

project planning
scheduling
risk management
resource management
monitoring
configuration management
quality management

Think:

```
People
+
Process
+
Time
+
Cost
+
Risk
+
Quality
```

12. Project Planning

Project planning determines:

what must be built
how it will be built
who will build it
how long it will take
how much it will cost
what risks exist

Outputs may include:

project scope
resource plan
cost estimates
schedule
risk plan

13. Project Scheduling

Scheduling determines **when activities occur**.

Important concepts:

tasks
dependencies
duration
milestones
deadlines
resources

Tools:

Gantt charts
PERT
network diagrams
critical path analysis

14. Risk Management

Risk = uncertain event that may negatively affect a project.

Examples:

requirements changing
key developer leaving
technology failure
budget shortage
schedule delay
security issue

Risk management cycle

```
Identify
↓
Analyze
↓
Prioritize
↓
Plan response
↓
Monitor
```

 **Memory**

Key Insight

I-A-P-M-M

>

Key Insight

Identify → Analyze → Prioritize → Mitigate → Monitor

MODULE II — SOFTWARE REQUIREMENTS ENGINEERING

12 Topics

Functional requirements
Non-functional requirements
User requirements
System requirements
SRS
IEEE SRS standard
Quality of good SRS
Feasibility study
Requirements elicitation
Requirements analysis
Requirements validation
Requirements management

15. Functional Requirements

Describe **what the system should do**.

Example:

Key Insight

The system shall allow users to reset their password.

Examples:

login
registration
payment
search
report generation

16. Non-Functional Requirements

Describe **how well** the system should work or constraints on it.

Examples:

performance
security

reliability
usability
scalability
availability
Example:

Key Insight

The system shall respond within 2 seconds.

Memory

Key Insight

Functional = WHAT

>

Key Insight

Non-functional = HOW WELL

17. User Requirements

High-level requirements written for customers/users.

Example:

Key Insight

Users should be able to book railway tickets online.

Generally understandable without technical implementation details.

18. System Requirements

Detailed technical requirements specifying what the system must provide.

Example:

Key Insight

The system shall authenticate a user using OTP within 30 seconds.

19. SRS

Software Requirements Specification

A formal document describing software requirements.

It acts as a bridge:

```
graph TD; A[Customer needs] --> B[SRS]; B --> C[Developers]; C --> D[System];
```

Typical SRS contents

introduction

scope

definitions

overall description

functional requirements

non-functional requirements

interfaces

constraints

assumptions

20. IEEE Standard for SRS

Your official syllabus explicitly includes the **IEEE standard for SRS**.

Know:

purpose

organization

characteristics

requirements description

interfaces

constraints

verification criteria

21. Characteristics of Good SRS

A good SRS should be:

correct

unambiguous

complete

consistent

verifiable

feasible

traceable

prioritized

modifiable

Memory

Key Insight

C-U-C-C-V-F-T-P-M

Don't just memorize the letters—understand what each means.

22. Feasibility Study

Determines whether a proposed project is practical.

Common dimensions:

Technical

Can we build it?

Economic

Is it financially worthwhile?

Operational

Will it work in the organization?

Legal

Does it comply with laws/regulations?

Schedule

Can we finish on time?

Memory

Key Insight

TELOS

>

Key Insight

Technical

Key Insight

Economic

Key Insight

Legal

Key Insight

Operational

Key Insight

Schedule

23. Requirements Elicitation

Collect requirements from stakeholders.

Techniques:

interviews
questionnaires
observation
workshops
brainstorming
document analysis
prototyping

24. Requirements Analysis

Analyze collected requirements to identify:

conflicts
ambiguity
missing requirements
dependencies
feasibility issues
priorities

25. Requirements Validation

Check whether requirements are suitable.

Validation checks:

validity
consistency
completeness
realism
verifiability

26. Requirements Management

Requirements change.

Therefore:

```
Requirement
↓
Track
↓
Change request
↓
Impact analysis
↓
Approval
↓
Update
↓
Traceability
```

MODULE III — SOFTWARE DESIGN ENGINEERING

13 Topics

Design engineering
Design process
Design quality
Design concepts
Design models
Object-oriented design
Cohesion
Coupling
UML use-case diagram
UML class diagram
UML activity diagram
UML sequence diagram
UML collaboration diagram

27. Design Engineering

Transforms requirements into an architecture and detailed design.

SRS

↓

Architecture

↓

Components

↓

Interfaces

↓

Detailed Design

↓

Code

28. Design Process

Typical steps:

understand requirements

identify architecture

identify components

design interfaces

design data

refine components

review design

29. Design Quality

Good software design should emphasize:

correctness

simplicity

modularity

maintainability

efficiency

reliability

reusability

testability

30. Design Concepts

Core concepts:

abstraction

modularity

information hiding

separation of concerns

refinement

architecture

cohesion

coupling

31. Design Models

Common design views:

```
Data/Class Model
+
Architectural Model
+
Interface Model
+
Component Model
+
Deployment Model
```

32. Object-Oriented Design

Models software using objects/classes.

Core concepts:

class

object

encapsulation

inheritance

polymorphism

abstraction

association

Example:

```
Class: Student

Attributes:
name
rollNo

Methods:
register()
attendClass()
```

33. Cohesion

Measures how strongly the elements of a module belong together.

High cohesion = GOOD

Example:

```
PaymentModule
├── validatePayment
├── processPayment
└── generateReceipt
```

All related.

🧠 Memory

Key Insight

Cohesion = inside

34. Coupling

Measures dependency between modules.

Low coupling = GOOD

```
Module A → interface → Module B
```

rather than tightly depending on internal details.

🧠 Perfect exam memory

Key Insight

High Cohesion + Low Coupling = Good Design

UML

Unified Modeling Language.

Used to visually represent software systems.

35. Use Case Diagram

Shows:

actors

system

use cases

relationships

Example:

```

Customer
|
|→ Login
|
|→ Place Order
|
|→ Track Order

```

36. Class Diagram

Shows:

classes

attributes

operations

relationships

Example:

```

+-----+
| Student |
+-----+
| name    |
| rollNo  |
+-----+
| register()
| attendClass()
+-----+

```

37. Activity Diagram

Represents workflow.

```

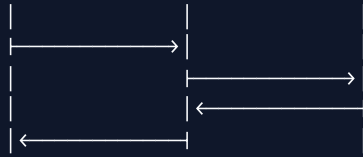
Start
↓
Login
↓
Valid?
├── No → Error
└── Yes
    ↓
    Dashboard
    ↓
    End

```

38. Sequence Diagram

Shows interactions **over time**.

User → LoginUI → AuthService → Database



Key:

Key Insight

vertical direction = time.

39. Collaboration Diagram

Focuses on interactions between objects and their relationships.

Unlike sequence diagrams, emphasis is more on **object organization and messages** than a vertical timeline.

MODULE IV — VERIFICATION, VALIDATION & TESTING

16 Topics

Verification

Validation

V&V planning

Software inspection

Static analysis

Software testing

Testing functions

Test-case design

White-box testing

Black-box testing

Basis-path testing

Control-structure testing

Unit testing

Integration testing

System testing

Reliability

40. Verification

Question:

Key Insight

Are we building the product right?

Checks whether artifacts conform to specifications.

Examples:

reviews

inspections

static analysis

41. Validation

Question:

Key Insight

Are we building the right product?

Checks whether software satisfies actual user needs.

🧠 **V&V memory**

Key Insight

Verification → specification

>

Key Insight

Validation → user

42. Verification & Validation Planning

Defines:

activities

responsibilities

schedules

standards

review strategy

testing strategy

acceptance criteria

43. Software Inspection

Formal review of software artifacts to find defects.

Can inspect:

requirements

design

code

documentation

Advantages:

Key Insight

defects can be found before execution.

44. Static Analysis

Analyze software **without executing it**.

Examples:

syntax checking

control-flow analysis

data-flow analysis

security analysis

code quality analysis

45. Software Testing

Dynamic execution of software to discover defects.

Basic flow:

```
Input
↓
Execute
↓
Observe output
↓
Compare expected vs actual
↓
Pass / Fail
```

46. Testing Functions

Testing should verify whether functions satisfy requirements.

Example:

For:

```
login(username,password)
```

test:

valid username/password

invalid username

invalid password

empty values

boundary cases

locked account

47. Test Case Design

A test case generally contains:

test ID

input

preconditions

steps

expected output

actual output

status

48. White-Box Testing

Tester knows internal implementation.

Focus:

statements

branches

conditions

paths

loops

49. Black-Box Testing

Tester focuses on external behavior without needing internal code.

Techniques:

equivalence partitioning

boundary value analysis

decision tables

state-transition testing

Key Insight

White = inside

>

Key Insight

Black = outside

50. Basis Path Testing

Uses control-flow structure to identify independent execution paths.

Important concept:

Cyclomatic complexity

\$\$

$$V(G) = E - N + 2$$

\$\$

where:

$\backslash(E)$ = edges

$\backslash(N)$ = nodes

For a connected graph with $\backslash(P)$ components:

\$\$

$$V(G) = E - N + 2P$$

\$\$

Exam importance

Very high.

You should be able to:

draw control-flow graph

calculate complexity

identify independent paths

derive test cases

51. Control Structure Testing

Tests internal control structures.

Includes:

condition testing

loop testing

data-flow testing

branch testing

52. Unit Testing

Tests individual components/modules.

Function

↓

Unit Test

↓

Pass/Fail

53. Integration Testing

Tests interactions among modules.

Strategies include:

top-down

bottom-up

sandwich

big-bang

54. System Testing

Tests the complete integrated system against requirements.

Includes:

functional testing

performance testing

security testing

recovery testing

usability testing

55. Reliability

Reliability = probability that software performs without failure under specified conditions for a specified period.

Important measures/concepts:

failure rate

MTTF

MTBF

availability

A common relationship:

\$\$

Availability=

$\frac{MTBF}{MTBF+MTTR}$

\$\$

MODULE V — SOFTWARE PROJECT MANAGEMENT & MAINTENANCE

13 Topics

Process metrics

Software measurement

Software project estimation

Decomposition techniques

Empirical estimation models

COCOMO

Function points

Quality assurance & standards

Quality planning

Quality control

Configuration management

Software maintenance

Software re-engineering

56. Process Metrics

Measure characteristics of the development process.

Examples:

effort

schedule

defect density

productivity

cycle time

rework

57. Software Measurement

Quantitative assessment of software/process attributes.

Possible measures:

Size
↓
Effort
↓
Cost
↓
Quality
↓
Productivity

58. Software Project Estimation

Estimate:

effort

time

cost

resources

Common approaches:

expert judgment

analogy

decomposition

empirical models

function points

COCOMO

59. Decomposition Techniques

Break a large problem into smaller pieces.

Two common approaches:

Problem decomposition

Break project into manageable tasks.

Process decomposition

Estimate individual activities and combine estimates.

60. Empirical Estimation Models

Use observed relationships from historical project data.

General form:

\$\$

$\text{Effort} = a(\text{Size})^b$

\$\$

The constants depend on the model/calibration.

61. COCOMO

Constructive Cost Model

A classic software estimation model associated with Barry Boehm.

Basic COCOMO:

\$\$

$\text{Effort} = a(\text{KLOC})^b$

\$\$

\$\$

$\text{Development Time} = c(\text{Effort})^d$

\$\$

where:

KLOC = thousand lines of code

coefficients depend on project mode

Basic project modes

Organic

Semi-detached

Embedded

Memory

Key Insight

COCOMO = Code → Cost → Months

62. Function Points

Estimate software size based on functionality delivered to users.

Five classic function types:

External Inputs

External Outputs

External Inquiries

Internal Logical Files

External Interface Files

Then:

```
Count functions
  ↓
Assign complexity weights
  ↓
Unadjusted FP
  ↓
Adjustment
  ↓
Function Points
```

Numerical topic

You should practice complete Function Point calculations.

63. Quality Assurance & Standards

Quality Assurance focuses on **process-oriented prevention**.

Standards provide consistent practices and expectations.

Examples of quality-related standards/frameworks may include:

ISO quality standards

software process standards

organizational coding/testing standards

64. Quality Planning

Determine:

quality objectives

standards

metrics

procedures

responsibilities

review/testing activities

65. Quality Control

Focuses on checking actual outputs against quality requirements.

Think:

```
Quality Planning
↓
Define standards
↓
Development
↓
Measure
↓
Compare
↓
Correct
```

QA vs QC

QA	QC
Process-oriented	Product/output-oriented
Prevent defects	Detect defects
Proactive	More reactive

66. Configuration Management

Controls changes to software artifacts.

Artifacts:

source code

requirements

design

test cases

documentation

Core activities:

```
Identify configuration items
↓
Version control
↓
Change control
↓
Status accounting
↓
Auditing
```

Memory

Key Insight

CM = Control Change

67. Software Maintenance

Software maintenance occurs after delivery.

Types:

Corrective

Fix defects.

Adaptive

Adapt to environmental changes.

Perfective

Improve functionality/performance.

Preventive

Improve maintainability/reduce future problems.

Memory

Key Insight

CAP-P

>

Key Insight

Corrective

Key Insight

Adaptive

Key Insight

Perfective

Key Insight

Preventive

Software Re-engineering

Improve/restructure an existing legacy system.

Possible activities:

```

Legacy System
↓
Analysis
↓
Reverse Engineering
↓
Restructuring
↓
Forward Engineering
↓
Modernized System

```

🔥 COMPLETE SE FORMULA SHEET

Cyclomatic Complexity

\$\$

$$V(G) = E - N + 2$$

\$\$

COCOMO Effort

\$\$

$$E = a(KLOC)^b$$

\$\$

COCOMO Development Time

\$\$

$$T = c(E)^d$$

\$\$

Productivity

\$\$

$$\text{Productivity} =$$

$$\frac{\text{Output}}{\text{Effort}}$$

\$\$

Availability

\$\$

$$\text{Availability} =$$

$$\frac{MTBF}{MTBF + MTTR}$$

\$\$

Basic Function Point structure

\$\$

$$FP = UFP \times VAF$$

\$\$

where:

UFP = Unadjusted Function Points

VAF = Value Adjustment Factor

ONE-PAGE MEMORY MAP



WHAT TO MASTER FIRST

For your exam preparation, I'd divide SE into four levels.

🔴 Tier S — Must be able to write/solve

Waterfall
Spiral
Agile
SRS
Functional vs non-functional requirements
Requirements engineering
Cohesion vs coupling
UML diagrams
Verification vs validation
White-box vs black-box testing
Basis Path Testing
Cyclomatic Complexity
Unit/Integration/System testing
COCOMO
Function Points
Software maintenance
Configuration management

🟡 Tier A — Strong theory

Prototyping
RAD
Incremental
Risk management
Feasibility study
Requirements validation
Design concepts
Static analysis
Control structure testing
Reliability
QA/QC
Re-engineering

🟡 Tier B

Definitions/FAQs
evolving role of software
process metrics
detailed management activities
detailed quality planning

🧠 ACTIVE-RECALL QUESTIONS

Don't reread everything repeatedly. Close the notes and answer these:

M1

What is software engineering?

Why is software engineering necessary?

Explain the Waterfall model.

When should a prototype be used?

Explain Spiral model.

Compare Waterfall and Agile.

Compare Incremental and Agile.

What is project scheduling?

Explain risk management.

M2

Functional vs non-functional requirements.

User vs system requirements.

What is SRS?

Characteristics of good SRS.

Explain feasibility study.

Explain requirement elicitation techniques.

Explain requirement validation.

Why is requirement management necessary?

M3

Explain software design process.

Explain abstraction and modularity.

Explain cohesion.

Explain coupling.

Why is high cohesion and low coupling desirable?

Draw a use-case diagram.

Draw a class diagram.

Draw an activity diagram.

Draw a sequence diagram.

Sequence vs collaboration diagram.

M4

Verification vs validation.

What is static analysis?

Explain software inspection.

White-box vs black-box testing.

Explain basis path testing.

Calculate cyclomatic complexity.

Explain unit testing.

Explain integration strategies.

Explain system testing.

Define software reliability.

M5

What are process metrics?

Explain software estimation.

Explain decomposition techniques.

Explain COCOMO.

Solve a COCOMO numerical.

Explain Function Points.

Solve a Function Point numerical.

QA vs QC.

Explain configuration management.

Explain four types of maintenance.

Explain software re-engineering.



THE MOST IMPORTANT CONNECTION

Instead of memorizing 67 isolated topics, remember the **software lifecycle**:

```
1. WHAT SHOULD WE BUILD?
  ↓
  REQUIREMENTS
  ↓
2. HOW SHOULD WE BUILD IT?
  ↓
  DESIGN
  ↓
3. HOW DO WE BUILD IT?
  ↓
  IMPLEMENTATION
  ↓
4. DOES IT WORK?
  ↓
  TESTING / V&V
  ↓
5. HOW DO WE MANAGE IT?
  ↓
  PROJECT
  ↓
6. HOW DO WE KEEP IT GOOD?
  ↓
  MAINTENANCE + CONFIGURATION
  ↓
7. HOW DO WE IMPROVE IT?
  ↓
  RE-ENGINEERING
```

That single chain gives you a **retrieval scaffold** for almost the entire subject.

The official syllabus confirms that these five modules collectively cover process models/project management, requirements engineering, design/UML, V&V/testing, and estimation/quality/configuration/maintenance.

One caveat: I found the official SE syllabus and the expanded 67-topic study checklist in your uploaded material, but I did **not** find SE-specific PYQ papers among the files currently searchable. So I have not fabricated "actual SE PYQs." If you upload the SE PYQs, I can map every question to these 67 topics and produce a **PYQ-frequency/VVI ranking** just like we did for NLP.